

ASTRA: AI-augmented Sprint Through Rapid Assembly

Phương pháp luận phát triển Scrum tăng cường AI dựa trên SPI

ASTRA là phương pháp luận **AI-Enhanced Scrum (Scrum tăng cường AI)** kết hợp hệ sinh thái AI agent của Claude Code vào quy trình Agile Scrum của phương pháp luận SPI (Success Path Integration), loại bỏ lãng phí còn tồn tại trong Scrum và tối đa hóa lợi thế của vibe coding.

Mục lục

- 1. [Tổng quan phương pháp luận](#)
- 2. [Sự tiến hóa từ Scrum sang ASTRA](#)
- 3. [Định nghĩa vai trò](#)
 - [Quy trình phát triển](#)
- 4. [Thiết lập ban đầu plugin](#)
- 5. [Xây dựng Design System](#)
- 6. [Viết Blueprint](#)
- 7. [Thiết kế cơ sở dữ liệu](#)
- 8. [Tạo Sprint dựa trên Blueprint](#)
- 9. [Triển khai](#)
- 10. [Viết kịch bản kiểm thử](#)
- 11. [Thực thi kiểm thử](#)
- 12. [PR / Review](#)
- 13. [Merge vào nhánh Staging](#)
- 14. [Kiểm thử người dùng](#)
- 15. [Merge vào nhánh Main](#)
 - [Phụ lục](#)
 - [A: Tham chiếu nhanh công cụ Claude Code](#)
 - [A-2: Tham chiếu nhanh Agent](#)
 - [B: Hướng dẫn viết Prompt](#)
 - [C: Quản lý rủi ro](#)
 - [D: Cơ sở ước lượng thời gian làm việc của AI Agent](#)
 - [E: Thiết lập dự án Sprint 0](#)
 - [F: Template dự án](#)
 - [G: Hiệu quả kỳ vọng](#)
 - [H: Hiệu quả chi phí](#)

1. Tổng quan phương pháp luận

ASTRA là gì?

AI-augmented **S**print **T**hrough **R**apid **A**ssembly

ASTRA mang ý nghĩa "ngôi sao (Astra, tiếng Latin)", đóng vai trò như la bàn dẫn dắt dự án nhanh chóng đến đích.

Triết lý cốt lõi

ASTRA **không phủ nhận Agile Scrum**. Đây là **Scrum tiến hóa** duy trì framework đã được kiểm chứng của Scrum, trong khi AI agent hấp thụ sự kém hiệu quả phát sinh trong mỗi hoạt động Scrum để tạo ra kết quả nhanh hơn và chất lượng cao hơn.

Điều đã thay đổi: Chu kỳ Sprint (2 tuần → 1 tuần, gia tăng nhỏ, phản hồi nhanh), tự động hóa công việc thủ công bằng AI (giảm 40~60% thời gian), Quality Gate tích hợp sẵn **Điều không thay đổi:** Chuyển giao giá trị dần dần của Scrum, 3 trụ cột Minh bạch - Kiểm tra - Thích ứng

Nguyên tắc VIP

Nguyên tắc	Cốt lõi	Công cụ thực hiện
Vibe-driven Development	Đừng viết code, hãy truyền đạt ý định	feature-dev, frontend-design
Instant Feedback Loop	Rút ngắn chu kỳ phản hồi trong Sprint xuống đơn vị giờ	chrome-devtools MCP, code-review
Plugin-powered Quality	Chất lượng được tích hợp vào code	astra-methodology, security-guidance, hookify

Mối quan hệ với SPI

5 giai đoạn SPI	Triển khai ASTRA	Công cụ sử dụng
1. Strategy	Product Vision + Xác minh công nghệ	context7 MCP
2. Process Map	Tình hình Product Backlog + Tự động tạo tài liệu thiết kế	feature-dev Phase 1-4
3. Iterative Build	Triển khai song song bằng AI (Sprint chu kỳ 1 tuần)	feature-dev + frontend-design
4. Integration	Xác minh tích hợp thời gian thực	chrome-devtools MCP
5. Success Launch	Tự động hóa tài liệu + Báo cáo chất lượng	feature-dev Phase 7

Nguyên tắc 3S của SPI	Triển khai ASTRA	Công cụ sử dụng
Standardization	Tự động áp dụng bắt buộc tại thời điểm viết	astra-methodology (PostToolUse hook)
Scalability	Tự động kiểm tra khả năng mở rộng	feature-dev code-architect
Security	Chặn mẫu bảo mật thời gian thực	security-guidance (PreToolUse hook)

2. Sự tiến hóa từ Scrum sang ASTRA

Phần này tổng hợp toàn bộ so sánh với Scrum truyền thống. Từ các phần tiếp theo sẽ tập trung vào phương pháp thực thi của chính ASTRA.

2.1 Tóm tắt thay đổi cốt lõi

Scrum truyền thống:

Product Backlog → Sprint Planning → Sprint(2 tuần) → Sprint Review → Retrospective

Phát triển → Kiểm thử → Review (thủ công, tuần tự)

ASTRA:

Product Backlog → Sprint Planning → Sprint(1 tuần) → Sprint Review → Retrospective

AI tinh chỉnh (code-explorer)

AI ước lượng (phân tích tự động)

AI thực thi song song (phát triển+ kiểm thử+review)

Demo thời gian thực (chrome-devtools)

AI phân tích (hookify)

2.2 So sánh theo hoạt động

Hoạt động	Scrum truyền thống	ASTRA	Cơ sở rút ngắn
Chu kỳ Sprint	2 tuần	1 tuần (gia tăng nhỏ, phản hồi nhanh)	AI xử lý song song phát triển+kiểm thử+review, chu kỳ ngắn tăng tính linh hoạt
Phân tích/Thiết kế Story	1~2 ngày	2~4 giờ	AI phân tích 20-40 phút + con người xem xét/bổ sung 4-2 giờ (feature-dev Phase 1-4)
Lập trình thủ công	5~7 ngày	2~4 ngày	AI tạo code 1-3 giờ + chu kỳ xác minh/sửa đổi của con người (giảm 40-60% dựa trên nghiên cứu METR)
Chờ Code Review	1~2 ngày	20~40 phút	AI review 10-15 phút + con người xem xét kết quả 4-20 phút (agent code-review song song)
Viết Unit Test	1~2 ngày	Xử lý đồng thời	feature-dev tạo test cùng code (cần xác minh của con người 30 phút~1 giờ)
Tranh luận coding standard	Lặp lại mỗi review	Ngăn chặn từ gốc	astra-methodology tự động áp dụng tại thời điểm viết
Kiểm tra bảo mật	Sprint riêng biệt	Chặn thời gian thực	security-guidance tự động chặn 9 mẫu
Bàn giao thiết kế UI	Chờ đợi Nhà thiết kế → Lập trình viên	Tạo trực tiếp	AI tạo 15-30 phút + DSA kiểm duyệt 4-12 giờ (frontend-design)
Hiệu quả hồi cứu	"Lần sau sẽ cải thiện"	Bắt buộc bằng quy tắc	Chuyển đổi thành quy tắc tự động ngay lập tức bằng hookify
Đối phó thay đổi yêu cầu	Sprint tiếp theo (2 tuần+)	1~2 ngày	Phân tích tác động + Sửa tài liệu thiết kế + AI phản ánh vào code + Xác minh của con người

2.3 So sánh Ceremony

Sự kiện	Thời gian truyền thống	Thời gian ASTRA	Nội dung tăng cường AI
Sprint Planning	4 giờ	1 giờ	Sử dụng báo cáo phân tích trước của feature-dev
Daily Scrum	15 phút x 10 ngày = 2.5h	Bắt đồng bộ	Báo cáo tiến độ tự động dựa trên commit
Design Review	(Không có riêng)	1 giờ	DSA kiểm duyệt UI do AI tạo
Sprint Review	2 giờ	1 giờ	Demo thời gian thực bằng chrome-devtools
Retrospective	1.5 giờ	30 phút	AI phân tích bằng sprint-analyzer → Tự động hóa bằng hookify
Backlog Refinement	2 giờ	30 phút	Phân tích tự động bằng feature-dev code-explorer
Tổng cộng	~12 giờ/Sprint	~4 giờ/Sprint	Giảm 67%

2.4 So sánh vai trò

Scrum truyền thống	ASTRA	Thay đổi
Product Owner (PO) 1 người	Domain Expert (DE) 1 người	Duy trì vai trò PO + phản hồi thời gian thực

Scrum Master (SM) 1 người	Vibe Architect (VA) 1 người	SM + kiến trúc + thiết kế prompt
Lập trình viên 3~5 người	Prompt Engineer (PE) 1~2 người	Lập trình thủ công → Thiết kế prompt + Xác minh
Nhà thiết kế UI 1 người	Design System Architect (DSA) 1 người	Xây dựng Design System + Kiểm duyệt
QA 1~2 người	(Thay thế bằng AI agent)	code-review + security-guidance
Tổng 7~10 người	Tổng 4~5 người	Giảm 50%

2.5 So sánh Artifact

Artifact Scrum	Hình thái tiến hóa ASTRA
Product Backlog	+ Liên kết Prompt Map docs/sprints/
Sprint Backlog	+ Prompt theo tính năng + Tài liệu thiết kế (MD)
Increment	+ Báo cáo chất lượng tự động + Living Document
Definition of Done (kiểm tra thủ công)	+ Xác minh tự động bằng AI Quality Gate (Gate 1-3)
(Tài liệu kiểm thử phân tán)	+ Quản lý tập trung chiến lược/test case/báo cáo kiểm thử tại docs/tests/
(Thiết kế DB phân tán)	+ Quản lý tập trung thiết kế DB/naming/migration tại docs/database/

2.6 Hiệu quả chi phí

Scrum truyền thống	ASTRA	Tiết kiệm
Thời gian: 5 tháng	3 tháng	40% ↓
Nhân sự: 8 người	4 người	50% ↓
Nhân công: 3.2 tỷ KRW	0.96 tỷ KRW	70% ↓
Chi phí API: -	0.07 tỷ KRW	-
Tổng chi phí: 3.5 tỷ KRW	1.1 tỷ KRW	69% ↓
※ Hiệu ứng nhân lên nhờ giảm đồng thời thời gian và nhân sự		
※ Tỷ lệ rút ngắn thời gian áp dụng 40~60% dựa trên nghiên cứu METR (tiêu chuẩn workflow AI có cấu trúc)		
※ Chất lượng thực tế được cải thiện nhờ AI Quality Gate tự động (tỷ lệ tuân thủ tiêu chuẩn 60~70% → 95%+)		

Bí quyết giảm đồng thời thời gian và nhân sự: AI agent hấp thụ **công việc thủ công lặp đi lặp lại** (lập trình, review, kiểm thử, kiểm tra tiêu chuẩn), nên con người chỉ tập trung vào **phán đoán và ra quyết định** (yêu cầu, kiến trúc, thiết kế, logic nghiệp vụ).

3. Định nghĩa vai trò

VA (Vibe Architect) - 1 Lập trình viên Senior

Mở rộng vai trò Scrum Master, đảm nhận thêm **điều phối AI agent**.

Năng lực cốt lõi:

- 1. **Prompt Engineering:** Chuyển đổi backlog item mơ hồ thành prompt chính xác
- 2. **Khả năng đánh giá kết quả AI:** Nhanh chóng đánh giá chất lượng/độ chính xác đầu ra của AI
- 3. **Tư duy kiến trúc:** Chọn phương án tối ưu trong nhiều phương án thiết kế từ `feature-dev` Phase 4
- 4. **Kiến thức domain:** Hiểu logic nghiệp vụ và truyền đạt chính xác cho AI

Hoạt động chính:

- Quản lý tiến trình Sprint + Thiết kế workflow AI agent
- Quản lý và tối ưu hóa chất lượng prompt
- Chuyển đổi kết quả hồi cứu thành quy tắc hookify
- Quyết định kiến trúc + Phán đoán cuối cùng Quality Gate

PE (Prompt Engineer) - 1~2 Lập trình viên Junior

Tập trung vào **viết prompt + xác minh kết quả AI** thay vì trực tiếp viết code.

Hoạt động chính:

- Viết prompt theo tính năng (dựa trên tài liệu thiết kế)
- Xác minh code và test do AI tạo
- Kiểm tra và xử lý kết quả AI review
- Xem xét và bổ sung tài liệu thiết kế (MD)

DE (Domain Expert) - 1 Người phụ trách nghiệp vụ bên khách hàng

Thêm **phương thức phản hồi thời gian thực** vào vai trò PO truyền thống.

Hoạt động chính:

- Truyền đạt yêu cầu trực tiếp bằng ngôn ngữ tự nhiên (cung cấp chất liệu cho prompt)
- Quản lý ưu tiên backlog
- Phản hồi ngay lập tức trong demo thời gian thực chrome-devtools
- Trực tiếp xác minh nghiệm thu trên hệ thống đang hoạt động

DSA (Design System Architect) - 1 Nhà thiết kế

Dù AI tạo code UI, **phán đoán chất lượng và tính nhất quán thiết kế vẫn cần nhà thiết kế chuyên nghiệp.**

Hoạt động chính:

- Sprint 0:** Xây dựng Design System (định nghĩa token màu sắc, typography, component, spacing)
- Sprint tính năng:** Kiểm duyệt thiết kế UI do AI tạo (xác nhận tuân thủ Design System)
- Release Sprint:** Kiểm duyệt thiết kế cuối cùng toàn bộ màn hình

Checklist kiểm duyệt thiết kế:

- ☐ Tuân thủ design token (màu sắc, font, spacing không lệch khỏi hệ thống token)
- ☐ Tính nhất quán component (cùng loại component không hiển thị khác nhau giữa các màn hình)
- ☐ Layout responsive (breakpoint mobile/tablet/desktop phù hợp)
- ☐ Đáp ứng accessibility cơ bản (độ tương phản màu, hiển thị focus, kích thước text)
- ☐ Tính nhất quán tương tác (thống nhất trạng thái hover/focus/active)
- ☐ Margin và căn chỉnh (tuân thủ hệ thống grid)

Quy trình phát triển

Chuẩn bị trước: Quy trình bên dưới giả định **Sprint 0 (thiết lập ban đầu dự án) đã hoàn tất**. Trong Sprint 0, thực hiện cấu hình môi trường phát triển (/astra-setup), tạo cấu trúc dự án (/project-init), xây dựng Design System, viết CLAUDE.md, cấu hình quy tắc hookify, v.v. Chi tiết tham khảo [Phụ lục E: Thiết lập dự án Sprint 0](#).

[Sprint 0]

Xây dựng Design System

[Sprint tính năng]

Viết Blueprint → Thiết kế CSDL → Tạo Sprint → Triển khai → Kịch bản kiểm thử → Thực thi kiểm thử → PR/Review



Merge nhánh Main ← Kiểm thử người dùng ← Merge nhánh

Staging ←

4. Thiết lập ban đầu plugin

Để sử dụng phương pháp luận ASTRA, trước tiên cần cài đặt **plugin astra-methodology** và cấu hình môi trường phát triển toàn cục. Quá trình này chỉ cần thực hiện **1 lần** trên mỗi máy lập trình viên.

Điều kiện tiên quyết: [Claude Code CLI](#) phải được cài đặt.

4.1 Cài đặt plugin astra-methodology

Thực hiện các lệnh sau theo thứ tự trong terminal.

```
# Bước 1: Đăng ký ASTRA marketplace
claude plugin marketplace add https://github.com/ASTRA-TECHNOLOGY-COMPANY-LIMITED/astra-methodology.git

# Bước 2: Cài đặt plugin astra-methodology
claude plugin install astra-methodology@astra
```

4.2 Thiết lập tự động môi trường phát triển toàn cục

Sau khi cài đặt plugin, chạy Claude Code và thực hiện lệnh `/astra-setup` để tự động cấu hình phần còn lại.

```
# Chạy Claude Code
claude

# Thực thi thiết lập môi trường phát triển toàn cục trong Claude Code
/astra-setup
```

Các tác vụ `/astra-setup` tự động thực hiện:

Lưu ý bảo mật: Chế độ `bypassPermissions` bỏ qua xác nhận sử dụng công cụ của Claude Code. Chỉ sử dụng trong môi trường đáng tin cậy.

1. Cấu hình toàn cục (`~/.claude/settings.json`)

- Kích hoạt biến môi trường Agent Teams
- Cấu hình chế độ Permission (`bypassPermissions`)
- Kích hoạt Always Thinking

2. Đăng ký MCP Server (`~/.claude/.mcp.json`)

- `chrome-devtools` — Dùng cho kiểm thử tích hợp trình duyệt
- `postgres` — Dùng cho kết nối cơ sở dữ liệu
- `context7` — Dùng cho truy vấn tài liệu thư viện mới nhất

3. Tự động cài đặt 9 plugin bắt buộc

- `claude-code-setup`, `code-review`, `code-simplifier`, `commit-commands`
- `feature-dev`, `frontend-design`, `hookify`, `security-guidance`, `context7`

4. Xác minh công cụ trước — Kiểm tra Node.js, npm/npv, Git, GitHub CLI đã cài đặt chưa

Sau khi thiết lập hoàn tất, báo cáo kết quả sẽ được xuất ra. Hãy xác nhận tất cả các mục đã được check.

5. Xây dựng Design System

Trong Sprint 0, DSA chủ trì xây dựng **Design System** cho dự án. Design System là nền tảng cốt lõi để AI tạo UI nhất quán, được quản lý tại `docs/design-system/`.

Nguyên tắc cốt lõi: UI do AI tạo mà không có Design System sẽ có style khác nhau ở mỗi màn hình. Hệ thống token đóng vai trò guardrail cho thiết kế của AI.

5.1 Cấu trúc thư mục Design System

```
docs/design-system/  
├─ design-tokens.css      # CSS Custom Properties (màu sắc, font, spacing)  
├─ tailwind.config.js     # Dùng cho dự án sử dụng Tailwind  
├─ components.md          # Hướng dẫn style component cốt lõi  
├─ layout-grid.md         # Hệ thống layout grid  
└─ references/            # Hình ảnh tham khảo thiết kế/moodboard
```

5.2 Định nghĩa Design Token

Design token định nghĩa các giá trị thiết kế như màu sắc, typography, spacing dưới dạng CSS Custom Properties.

```
# Tạo file design token  
/feature-dev "Định nghĩa design token cho dự án tại docs/design-system/design-tokens.css.  
- Bảng màu (Primary, Secondary, Neutral, Semantic)  
- Typography (Font Family, Size Scale, Weight, Line Height)  
- Spacing (grid 4px: 4, 8, 12, 16, 24, 32, 48, 64)  
- Breakpoint (Mobile: 375px, Tablet: 768px, Desktop: 1024px, Wide: 1440px)  
- Shadow, border-radius, transition  
Chưa sửa code."
```

5.3 Hướng dẫn Style Component

Tài liệu hóa quy cách thiết kế của các UI component cốt lõi.

```
# Viết hướng dẫn style component  
/feature-dev "Viết hướng dẫn style component cốt lõi tại docs/design-system/components.md.  
- Button (Primary, Secondary, Ghost, Danger – mỗi trạng thái: default, hover, active, disabled)  
- Input (Text, Password, Search, TextArea – trạng thái: default, focus, error, disabled)  
- Card, Modal, Toast/Alert  
- Navigation (Header, Sidebar, Breadcrumb, Tab)  
- Table, Pagination  
- Mỗi component chỉ sử dụng token từ design-tokens.css  
Chưa sửa code."
```

5.4 Hệ thống Layout Grid

```
# Định nghĩa hệ thống layout grid  
/feature-dev "Định nghĩa hệ thống layout grid tại docs/design-system/layout-grid.md.  
- Grid 12 cột (gutter: 16px mobile, 24px desktop)  
- Mẫu layout trang (Sidebar + Content, Full-width, Centered)  
- Quy tắc responsive (mobile first)  
- Chiều rộng tối đa container  
Chưa sửa code."
```

5.5 Tạo trang Preview Design System

Khi design token, component, layout grid đã được định nghĩa, tạo **trang preview có thể kiểm tra thực tế trên trình duyệt**. Vì chỉ bằng tài liệu khó có thể đánh giá chính xác màu sắc, typography, trạng thái component, nên cần một trang cho DSA và toàn đội kiểm tra trực quan.

Tại sao cần trang preview?

- Kiểm tra giá trị design token (màu, font, spacing) bằng **kết quả render thực tế**
- Xác minh **tương tác** các trạng thái component (default, hover, active, disabled)
- DSA **kiểm thử ngay lập tức** responsive/accessibility bằng `chrome-devtools` MCP
- Đóng vai trò **điểm chuẩn (Baseline)** cho UI mà AI tạo trong Sprint tính năng

```
# Tạo trang preview Design System
/frontend-design "Tạo trang preview Design System để xem tổng quan design token,
hướng dẫn style component, layout grid từ docs/design-system/.
- Swatch bảng màu (toàn bộ Primary, Secondary, Neutral, Semantic)
- Xem trước tỷ lệ typography (mỗi tổ hợp size/weight)
- Trực quan hóa hệ thống spacing (block theo đơn vị grid 4px)
- Showcase component cốt lõi (Button, Input, Card, Modal, Toast – tất cả trạng thái)
- Overlay layout grid (trực quan hóa grid 12 cột)
- Xem trước theo breakpoint responsive
- Chỉ sử dụng token từ docs/design-system/design-tokens.css"
```

Xác minh trang preview (DSA chủ trì):

- Kiểm tra render ở mỗi viewport (375px, 768px, 1024px, 1440px) bằng `chrome-devtools` MCP
- Kiểm tra các mục accessibility cơ bản như tỷ lệ tương phản màu, hiển thị focus
- Khi phát hiện vấn đề, sửa design token hoặc hướng dẫn component và phản ánh ngay vào trang preview

5.6 Checklist hoàn thành Design System

- ☐ Hoàn thành định nghĩa bảng màu (tỷ lệ tương phản accessibility 4.5:1 trở lên)
- ☐ Hoàn thành định nghĩa tỷ lệ typography
- ☐ Hoàn thành định nghĩa hệ thống spacing (dựa trên 4px hoặc 8px)
- ☐ Hoàn thành viết hướng dẫn style component cốt lõi
- ☐ Hoàn thành định nghĩa hệ thống layout grid
- ☐ **Hoàn thành tạo trang preview Design System và DSA đã xác minh**
- ☐ Hoàn thành thu thập tham khảo thiết kế/moodboard (nếu có)

6. Viết Blueprint

Trước khi triển khai tính năng, **viết tài liệu thiết kế (Blueprint)** trước. Blueprint là đầu vào cốt lõi để AI tạo code chính xác, được quản lý theo đơn vị **thư mục được đánh số** tại `docs/blueprints/` .

Cấu trúc thư mục: Mỗi blueprint tính năng được tổ chức dạng thư mục `docs/blueprints/{NNN}-{feature-name}/` (ví dụ: `001-auth/` , `002-payment/`). Tài liệu thiết kế chính là `blueprint.md` , các file phụ trợ liên quan (diagram, API spec, v.v.) đặt trong cùng thư mục.

Nguyên tắc cốt lõi: Blueprint tốt = Code tốt. Chất lượng spec quyết định chất lượng đầu ra AI. (Có spec: ~~12 giờ, không spec:~~ 48 giờ+ — tham khảo [Phụ lục D](#))

6.1 Viết tài liệu thiết kế tính năng

```
# Tự động tạo tài liệu thiết kế tính năng cốt lõi
/feature-dev "Viết tài liệu thiết kế hệ thống xác thực người dùng dựa trên JWT
tại docs/blueprints/001-auth/blueprint.md."
```


- Bao gồm chức năng đăng ký, đăng nhập, làm mới token, quản lý quyền (RBAC)
 - Mật khẩu hash bằng bcrypt
 - Access Token hết hạn sau 30 phút, Refresh Token 7 ngày
 - DB schema tham chiếu docs/database/database-design.md
- Chưa sửa code."

→ VA/DE mở trực tiếp docs/blueprints/001-auth/blueprint.md để xem xét và chỉnh sửa

→ Tiến hành bước tiếp theo sau khi DE phê duyệt

6.2 Checklist hoàn thành Blueprint

- ☐ Hoàn thành viết tài liệu thiết kế tính năng (docs/blueprints/{NNN}-{feature-name}/blueprint.md)
- ☐ DE đã phê duyệt

7. Thiết kế cơ sở dữ liệu

Khi Blueprint hoàn tất, **phản ánh các bảng cơ sở dữ liệu cần thiết vào tài liệu thiết kế DB tập trung**. Tất cả thiết kế bảng được quản lý trong một tài liệu duy nhất docs/database/database-design.md .

Tại sao một tài liệu duy nhất?

- **AI nhận biết cùng lúc** toàn bộ cấu trúc bảng và mối quan hệ để thiết kế có tính nhất quán
- **Xác minh tham chiếu FK** giữa các bảng, trùng lặp cột, nhất quán naming trong một ngữ cảnh duy nhất

7.1 Phản ánh vào tài liệu thiết kế DB

```
# Thiết kế bảng DB dựa trên Blueprint
/feature-dev "Phân tích blueprint docs/blueprints/001-auth/blueprint.md
và thiết kế các bảng cơ sở dữ liệu cần thiết vào docs/database/database-design.md.
- Trích xuất các bảng, cột, quan hệ cần thiết từ yêu cầu tính năng của blueprint
- Phản ánh quan hệ FK với các bảng hiện có vào phần ERD và tóm tắt quan hệ
- Tuân thủ từ điển thuật ngữ chuẩn (sử dụng /lookup-term)
Chưa sửa code."

# Cũng có thể chỉ định trực tiếp bảng để phản ánh
/feature-dev "Thêm/cập nhật các bảng module xác thực vào docs/database/database-design.md:
- TB_COMM_USER (người dùng), TB_COMM_TRMS (điều khoản), TH_COMM_USER_AGRE (lịch sử đồng ý)
- Phản ánh quan hệ FK với các bảng hiện có vào phần ERD và tóm tắt quan hệ
- Tuân thủ từ điển thuật ngữ chuẩn (sử dụng /lookup-term)
Chưa sửa code."

# Tra cứu thuật ngữ chuẩn
/lookup-term 결제금액
/lookup-term 주문번호
```

7.2 Áp dụng tiêu chuẩn mã quốc tế (nếu có)

Áp dụng khi có tính năng cần mã quốc tế như nhập số điện thoại, bộ chọn quốc gia/vùng, biểu mẫu địa chỉ.

Tiêu chuẩn	Mục đích	Quy tắc cột DB
ISO 3166-1 (alpha-2)	Mã quốc gia (KR, US, JP)	NATN_CD CHAR(2)
ISO 3166-2	Mã vùng (KR-11, US-CA)	RGN_CD VARCHAR(6)
ITU-T E.164	Số điện thoại quốc tế (+821012345678)	INTL_TELNO VARCHAR(15)

```
/lookup-code KR
/lookup-code US-CA
/lookup-code +82
```

7.3 Viết SQL Migration

```
# Ghi nhận SQL migration
/feature-dev "Viết SQL migration cho các bảng module đơn hàng mới thêm
từ docs/database/database-design.md vào docs/database/migration/v1.1.0-order.sql.
- Bao gồm câu lệnh CREATE TABLE + Index + FK constraint
- Viết cả SQL rollback
Chưa áp dụng vào DB thực."
```

7.4 Checklist hoàn thành thiết kế cơ sở dữ liệu

- ☐ Hoàn thành phản ánh bảng vào tài liệu thiết kế DB (docs/database/database-design.md)
- ☐ Xác nhận tuân thủ từ điển thuật ngữ chuẩn (/lookup-term)
- ☐ Xác nhận áp dụng tiêu chuẩn mã quốc tế (nếu có)
- ☐ Hoàn thành viết SQL migration

8. Tạo Sprint dựa trên Blueprint

Khi Blueprint hoàn tất, lập kế hoạch Sprint dựa trên đó. Khởi tạo tài liệu Sprint bằng lệnh `/sprint-plan` và phân bổ các tính năng từ Blueprint vào Sprint backlog.

8.1 Khởi tạo Sprint

```
# Tạo tài liệu Sprint (prompt map, theo dõi tiến độ, template hồi cứu)
/sprint-plan 1
```

Các file được tạo:

- `docs/sprints/sprint-1/prompt-map.md` — Kế hoạch prompt theo tính năng
- `docs/sprints/sprint-1/progress.md` — Bảng theo dõi tiến độ
- `docs/sprints/sprint-1/retrospective.md` — Template hồi cứu

8.2 Sprint Planning (1 giờ)

Chuẩn bị trước (Trước ngày Planning, VA thực hiện)

```
/feature-dev "Phân tích độ phức tạp kỹ thuật của các backlog item ứng viên Sprint này:
1. Xác thực người dùng (OAuth 2.0 + JWT)
2. Dashboard thanh toán
3. Trang cài đặt thông báo
Tổng hợp phụ thuộc với codebase hiện có, quy mô công việc dự kiến, yếu tố rủi ro.
Chưa sửa code."
```

Cuộc họp Planning (1 giờ)

Thời gian	Hoạt động	Người tham gia
10 phút	Review báo cáo phân tích AI (thay thế ước lượng story point)	VA, PE
20 phút	Xác nhận ưu tiên kinh doanh với DE và thống nhất mục tiêu Sprint	DE, VA

20 phút	Thảo luận hướng thiết kế prompt theo item + DSA chia sẻ hướng thiết kế	VA, PE, DSA
10 phút	Xác định Sprint backlog	Toàn bộ

8.3 Viết Prompt Map

Phân tách mỗi tính năng từ Blueprint thành đơn vị prompt và ghi vào `prompt-map.md`.

```
# Sprint 1 Prompt Map

## Mục tiêu Sprint
[Mô tả giá trị kinh doanh cần đạt được trong Sprint này]

## Tính năng 1: Xác thực người dùng
### 1.1 Tham chiếu Blueprint
- docs/blueprints/001-auth/blueprint.md
- docs/database/database-design.md (module xác thực)

### 1.2 Prompt triển khai
/feature-dev "Tuần thủ nghiêm ngặt nội dung
docs/blueprints/001-auth/blueprint.md và
docs/database/database-design.md để tiến hành phát triển."

## Tính năng 2: Dashboard thanh toán
### 2.1 Tham chiếu Blueprint
- docs/blueprints/002-payment-dashboard/blueprint.md

### 2.2 Prompt triển khai
/feature-dev "Tuần thủ nghiêm ngặt nội dung
docs/blueprints/002-payment-dashboard/blueprint.md và
docs/database/database-design.md để tiến hành phát triển."
```

8.4 Backlog Refinement (30 phút)

```
# Phân tích AI trước (trước Refinement)
/feature-dev "Phân tích các backlog item sau:
1. Quy trình hủy đơn/hoàn tiền
2. Trang thống kê dashboard quản trị
3. Quản lý cài đặt thông báo người dùng
Tổng hợp mối liên quan với codebase hiện có, rủi ro kỹ thuật, điều kiện tiên quyết.
Chưa sửa code."

# Cuộc họp Refinement (30 phút)
# └─ Review kết quả phân tích AI
# └─ Xác nhận giá trị kinh doanh/ưu tiên với DE
# └─ Phân tách item nếu cần
```

9. Triển khai

Triển khai code thực tế theo Sprint prompt map. Trong quá trình triển khai, **Gate 1 (thời điểm viết)** Quality Gate được tự động áp dụng.

9.1 Triển khai dựa trên tài liệu thiết kế

```
/feature-dev "Tuần thử nghiệm ngặt nội dung
docs/blueprints/001-auth/blueprint.md và
docs/database/database-design.md để tiến hành phát triển.
Test tham chiếu docs/tests/test-cases/sprint-1/auth-test-cases.md
để viết, và khi triển khai xong hãy chạy tất cả test
rồi báo cáo kết quả tại docs/tests/test-reports/."
```

Những gì được tự động thực thi với một dòng prompt này:

1. `code-explorer` phân tích codebase hiện có (2~3 song song)
2. Câu hỏi làm rõ (edge case, xác nhận quy tắc nghiệp vụ)
3. `code-architect` đề xuất kế hoạch triển khai (2~3 song song)
4. Viết code sau khi được phê duyệt
 - o `astra-methodology` tự động kiểm tra từ cấm/naming (PostToolUse hook)
 - o `security-guidance` tự động chặn mẫu bảo mật (PreToolUse hook)
 - o `Skill coding-convention` tự động áp dụng convention
5. `code-reviewer` kiểm tra chất lượng (3 song song)
6. Tạo tài liệu tóm tắt hoàn thành

9.2 Triển khai UI

Khi yêu cầu công việc frontend, skill `frontend-design` tự động kích hoạt để tạo UI cấp production.

```
# Chỉ định hướng thẩm mỹ sẽ cho kết quả tốt hơn
"Tạo dashboard thanh toán.
- Tình hình thanh toán thời gian thực (số lượng/số tiền hôm nay)
- Biểu đồ doanh thu theo ngày (30 ngày gần nhất)
- Danh sách giao dịch gần đây (phân trang)
- Dark mode mặc định, phong cách minimalist
- Bắt buộc sử dụng hệ thống token từ docs/design-system/design-tokens.css"

# Ví dụ chỉ định hướng thẩm mỹ đa dạng
"Tạo trang portfolio phong cách brutalist"
"Tạo trang chi tiết sản phẩm sang trọng phong cách art deco"
```

9.3 Xác minh thời gian thực (chrome-devtools MCP)

```
# Kiểm tra layout
"Chụp snapshot trang hiện tại và kiểm tra layout"

# Xác minh hoạt động API
"Kiểm tra xem API call có được thực hiện bình thường không bằng network request"

# Kiểm tra lỗi
"Kiểm tra xem có lỗi trong console không"

# Kiểm tra responsive (chuyển đổi viewport)
"Chuyển sang viewport mobile (375x667) và kiểm tra layout"
```

9.4 Tham chiếu API mới nhất (context7 MCP)

```
"use context7 - Cách thực hiện HTTP request bất đồng bộ bằng WebClient trong Spring Boot 3"
"use context7 - Cách sử dụng transaction trong Prisma"
"use context7 - Cú pháp Server Actions trong Next.js 15"
```

9.5 Commit

```
# Commit mỗi khi hoàn thành tính năng
/commit
```

9.6 Gate 1: WRITE-TIME (Tự động áp dụng)

Quality Gate tự động áp dụng cho mọi thao tác viết code (Write/Edit) trong quá trình triển khai.

Công cụ	Nội dung kiểm tra	Cách hoạt động
security-guidance	9 mẫu bảo mật (eval, innerHTML, v.v.)	PreToolUse hook, chặn (exit 2)
astra-methodology	Từ cấm + Quy tắc naming	PostToolUse hook, cảnh báo (exit 0)
hookify	Quy tắc tùy chỉnh theo dự án	PreToolUse/PostToolUse hook
Skill coding-convention	Tự động áp dụng convention Java/TS/RN/Python/CSS/SCSS	Skill (tự động phát hiện)
Skill data-standard	Áp dụng từ điển thuật ngữ chuẩn dữ liệu công cộng (공공데이터 표준 용어 사전)	Skill (tự động phát hiện khi code DB)
Skill code-standard	Áp dụng tiêu chuẩn ISO 3166-1/2, ITU-T E.164	Skill (tự động phát hiện khi số điện thoại/quốc gia/địa chỉ)

9.7 Đối phó thay đổi yêu cầu

Khi thay đổi yêu cầu phát sinh giữa Sprint, thực hiện theo quy trình sau.

```
# 1. Phân tích tác động (30 phút~1 giờ)
/feature-dev "Có yêu cầu thêm 'Thanh toán nhanh (KakaoPay)' vào phương thức thanh toán.
Tham chiếu codebase hiện có và docs/database/database-design.md
để phân tích phạm vi ảnh hưởng của module thanh toán.
Chưa sửa code."

# 2. Sửa Blueprint (1~2 giờ)
# → Thêm phần thanh toán nhanh vào docs/blueprints/003-payment/blueprint.md
# → Phản ánh thay đổi bằng vào docs/database/database-design.md

# 3. Phản ánh vào code (4~8 giờ)
/feature-dev "Phản ánh nội dung cập nhật từ
docs/blueprints/003-payment/blueprint.md và
docs/database/database-design.md để triển khai
chức năng thanh toán nhanh (KakaoPay).
Sử dụng mẫu PaymentProvider để không ảnh hưởng logic thanh toán hiện có."

# 4. Xác minh chất lượng tự động (30 phút~1 giờ)
/code-review
```

10. Viết kịch bản kiểm thử

Tạo kịch bản kiểm thử E2E dựa trên các tính năng đã triển khai trong Sprint. Lệnh `/test-scenario` phân tích blueprint, thiết kế DB, route, API endpoint để tự động viết kịch bản kiểm thử toàn diện.

10.1 Tạo kịch bản kiểm thử E2E

```
# Tự động tạo kịch bản E2E dựa trên blueprint, DB, route
/test-scenario
```

Các mục `/test-scenario` tự động phân tích:

- `docs/blueprints/{NNN}-{feature-name}/` — Yêu cầu tính năng
- `docs/database/database-design.md` — Mô hình dữ liệu
- `Route/API endpoint` — Luồng màn hình
- `Code test hiện có` — Kịch bản bị thiếu

10.2 Ví dụ: Viết kịch bản kiểm thử Sprint 1

Ví dụ viết kịch bản kiểm thử bằng lệnh `/test-scenario` sau khi hoàn thành triển khai tính năng xác thực trong Sprint 1.

```
# Tự động tạo kịch bản kiểm thử Sprint 1
/test-scenario Viết kịch bản kiểm thử cho Sprint 1.

# → Các tác vụ /test-scenario tự động thực hiện:
# 1. Quét docs/blueprints/{NNN}-*/ — Thu thập yêu cầu tính năng Sprint 1
# 2. Phân tích docs/database/database-design.md — Nắm bắt cấu trúc bảng liên quan
# 3. Khám phá route/API endpoint trong src/ — Ánh xạ luồng màn hình và đường dẫn API
# 4. Kiểm tra code test hiện có — Xác định kịch bản bị thiếu
#
# → Kết quả tạo: Tạo tài liệu kịch bản kiểm thử tại docs/tests/test-cases/sprint-1/
#   - Kịch bản E2E (luồng Đăng ký→Đăng nhập→Làm mới token→Kiểm tra quyền)
#   - Test case theo tính năng (định dạng Given-When-Then)
#   - Edge case và kịch bản lỗi
```

11. Thực thi kiểm thử

Thực hiện kiểm thử thực tế dựa trên kịch bản kiểm thử. Lệnh `/test-run` tự động thực thi chạy server + kiểm thử tích hợp Chrome MCP.

11.1 Thực thi kiểm thử tích hợp

```
# Tự động thực thi chạy server + kiểm thử tích hợp Chrome MCP
/test-run

# → Tự động chạy server + Giám sát log
# → Xác minh trang (snapshot, layout)
# → Kiểm tra hoạt động API (network request)
# → Đo hiệu suất (Core Web Vitals)
# → Kiểm tra lỗi console
```

11.2 Xác minh chi tiết thủ công

```
# Kiểm thử tích hợp API
"Kiểm thử tích hợp giữa API thanh toán và API đơn hàng. Giám sát network request và xác minh response."

# Kiểm tra tính nhất quán dữ liệu DB
"Kiểm tra xem định nghĩa quan hệ FK trong docs/database/database-design.md có khớp với DB schema thực tế không"
```

```
# Profiling hiệu suất
"Chạy performance trace toàn bộ trang và phân tích điểm nghẽn"

# Kiểm thử cross browser/responsive
"Chuyển sang viewport mobile (375x667) và kiểm tra layout"
"Chuyển sang viewport tablet (768x1024) và kiểm tra"
```

11.3 Báo cáo kết quả kiểm thử

```
/feature-dev "Viết báo cáo kết quả kiểm thử toàn bộ tại docs/tests/test-reports/sprint-1-report.md.
Bao gồm các nội dung sau:
- Tình trạng pass/fail kiểm thử theo module
- Tóm tắt test coverage
- Vấn đề phát hiện và biện pháp xử lý
- Tỷ lệ đạt được so với mục tiêu trong docs/tests/test-strategy.md"
```

11.4 Ví dụ: Thực thi kiểm thử Sprint 1

Ví dụ toàn bộ luồng thực hiện kiểm thử thực tế sau khi kịch bản kiểm thử tính năng xác thực Sprint 1 được viết xong.

Step 1: Thực thi tự động kiểm thử tích hợp

```
# Tự động thực thi chạy server + kiểm thử tích hợp Chrome MCP
/test-run

# → Luồng thực thi tự động:
# 1. Tự động chạy server + Giám sát log
# 2. Truy cập trang đăng ký → Nhập form → Gửi → Xác nhận thành công
# 3. Truy cập trang đăng nhập → Xác thực → Xác nhận phát hành token
# 4. Kiểm tra network request (xác minh response POST /auth/signup, POST /auth/login)
# 5. Xác nhận 0 lỗi console
# 6. Đo hiệu suất (Core Web Vitals)
```

Step 2: Xác minh chi tiết thủ công

```
# Xác minh hoạt động API endpoint xác thực
"Kiểm thử tuần tự luồng Đăng ký → Đăng nhập → Làm mới token.
Kiểm tra network request và response ở mỗi bước rồi cho biết kết quả."

# Xác minh edge case
"Thử đăng nhập với mật khẩu sai. Kiểm tra xem error response có đúng không."
"Gọi API được bảo vệ với Access Token hết hạn. Kiểm tra xem có nhận response 401 không."

# Kiểm tra responsive (form đăng nhập/đăng ký)
"Chuyển sang viewport mobile (375x667) và kiểm tra layout trang đăng nhập"

# Kiểm tra tính nhất quán dữ liệu DB
"Kiểm tra xem dữ liệu có được nhập bình thường vào bảng TB_COMM_USER sau khi đăng ký không"
```

Step 3: Viết báo cáo kết quả kiểm thử

```
/feature-dev "Viết báo cáo kết quả kiểm thử toàn bộ tại docs/tests/test-reports/sprint-1-report.md.
Bao gồm các nội dung sau:
- Tình trạng pass/fail kiểm thử module xác thực"
```

```
- Tóm tắt test coverage (mục tiêu: 70%+)
- Vấn đề phát hiện và biện pháp xử lý
- Tỷ lệ đạt được so với mục tiêu trong docs/tests/test-strategy.md"

# → Ví dụ kết quả tạo (docs/tests/test-reports/sprint-1-report.md):
#
# ## Tóm tắt kết quả kiểm thử
# | Module          | Tổng | Pass | Fail | Coverage |
# |-----|-----|-----|-----|-----|
# | Xác thực       | 15   | 14   | 1    | 82%      |
# | Quyền (RBAC)   | 8    | 8    | 0    | 78%      |
#
# ## Vấn đề phát hiện
# - ISS-001: Thông báo lỗi chung khi Refresh Token hết hạn → Đã sửa
#
# ## Tỷ lệ đạt được so với mục tiêu
# - Mục tiêu coverage 70% → Thực tế 80% ✓
# - Cover 100% kịch bản rủi ro cao ✓
```

12. PR / Review

Khi triển khai và kiểm thử hoàn tất, tạo PR và thực hiện code review. Lệnh `/pr-merge` xử lý hàng loạt commit→tạo PR→review→sửa→merge.

12.1 Tạo PR + Code Review

```
# Phương pháp 1: Chu kỳ tự động hóa toàn bộ (commit→PR→review→sửa→merge)
/pr-merge

# Phương pháp 2: Thực hiện thủ công từng bước
/commit-push-pr          # Commit + Push + Tạo PR
/code-review             # Code review song song 5 agent (chỉ báo cáo issue có độ tin cậy cao 80+)
```

12.2 Design Review (DSA chủ trì)

Khi có tính năng UI, DSA thực hiện kiểm duyệt thiết kế.

```
[Design Review]
├─ DSA kiểm tra màn hình thực tế bằng chrome-devtools MCP
│  └─ Kiểm tra tuân thủ design token
│     └─ Kiểm tra layout responsive (chuyển đổi viewport)
│        └─ Kiểm tra accessibility cơ bản
└─ Sửa issue
   └─ DSA: "Màu button này khác với token", "Margin không khớp grid 8px"
   └─ PE: Phản ánh feedback thiết kế vào prompt → AI tạo lại (5~10 phút)
   └─ DSA: Kiểm tra kết quả sửa ngay lập tức → Phê duyệt
```

12.3 Gate 2: REVIEW-TIME

Công cụ	Nội dung kiểm tra
feature-dev (code-reviewer tích hợp)	Chất lượng code/Bug/Convention (3 agent song song)
/code-review	Tuân thủ CLAUDE.md, Bug, Phân tích lịch sử (5 agent song song, lọc 80+)

Agent blueprint-reviewer	Xác minh chất lượng/nhất quán tài liệu thiết kế (Sonnet, chỉ đọc)
Agent test-coverage-analyzer	Phân tích chiến lược/coverage kiểm thử (Haiku, chỉ đọc)
Agent convention-validator	Xác minh coding convention (Haiku, chỉ đọc)

12.4 Gate 2.5: DESIGN-TIME (DSA kiểm duyệt)

Hạng mục kiểm duyệt	Phương pháp kiểm tra
Tuân thủ design token	Snapshot chrome-devtools + Agent design-token-validator (Haiku, xác minh tự động)
Tính nhất quán component	So sánh giữa các màn hình
Layout responsive	Chuyển đổi viewport chrome-devtools
Kiểm tra accessibility cơ bản	Kiểm tra tương phản màu, focus

Khi phát hiện issue: Feedback DSA → PE sửa prompt → AI tạo lại → DSA kiểm duyệt lại (hoàn thành trong 1 giờ)

12.5 Kiểm tra chất lượng bổ sung

```
/check-convention src/      # Kiểm tra coding convention
/check-naming src/entity/    # Kiểm tra tiêu chuẩn naming DB
```

12.6 Ví dụ: Thực thi PR và Review Sprint 1

Ví dụ toàn bộ luồng từ tạo PR đến merge sau khi hoàn thành triển khai tính năng xác thực Sprint 1.

Step 1: Commit + Tạo PR + Code Review + Merge (tự động hóa)

```
# Tự động thực thi toàn bộ chu kỳ bằng một lệnh /pr-merge
/pr-merge

# → Luồng thực thi tự động:
# 1. Commit thay đổi (tự động tạo commit message)
# 2. Push nhánh tính năng (feature/sprint-1-auth → origin)
# 3. Tạo PR (Triển khai tính năng xác thực Sprint 1)
# 4. Code review (5 agent song song – chỉ báo cáo issue có độ tin cậy cao 80+)
# 5. Tự động sửa issue phát hiện
# 6. Review lại → Merge khi pass
```

Step 2: Thực hiện thủ công từng bước (khi cần kiểm soát chi tiết)

```
# Bước 1: Commit + Push + Tạo PR
/commit
git push -u origin feature/sprint-1-auth
gh pr create --title "feat: Sprint 1 사용자 인증 구현" --body "## Summary
- JWT 기반 회원가입/로그인/토큰갱신 구현
- RBAC 권한 관리
- docs/blueprints/001-auth/blueprint.md 설계 준수

## Test plan
- [ ] 단위 테스트 통과 확인
- [ ] API 통합 테스트 확인
- [ ] 보안 패턴 검사 통과"
```

```
# Bước 2: Code review (5 agent song song)
/code-review

# Bước 3: Kiểm tra kết quả review rồi sửa issue
# → Chỉ báo cáo issue có độ tin cậy cao (80+) nên tập trung vào mục quan trọng

# Bước 4: Kiểm tra chất lượng
/check-convention src/
/check-naming src/entity/

# Bước 5: Commit sửa đổi + Review lại
/commit
/code-review

# Bước 6: Merge
gh pr merge --squash
```

Step 3: Design Review (khi bao gồm UI, DSA chủ trì)

```
# DSA kiểm tra màn hình thực tế bằng chrome-devtools MCP
"Chụp snapshot trang đăng nhập và kiểm tra tuân thủ design token"
"Chuyển sang viewport mobile (375x667) và kiểm tra layout form đăng nhập"

# Phản ánh feedback DSA
# → "Màu trạng thái lỗi trường nhập mật khẩu khác với token"
# → PE sửa prompt → AI tạo lại (5~10 phút) → DSA kiểm duyệt lại
```

Step 4: Ví dụ kết quả xác minh chất lượng Gate 2

[Kết quả Code Review – Tính năng xác thực Sprint 1]

code-reviewer (3 agent)	✅ Pass
convention-validator	✅ 0 vi phạm
blueprint-reviewer	✅ Khớp thiết kế
test-coverage-analyzer	✅ Coverage 82%
security-guidance	✅ 0 issue bảo mật

→ Toàn bộ Gate 2 Pass – Có thể merge vào Staging

13. Merge vào nhánh Staging

Merge nhánh tính năng đã pass kiểm thử vào nhánh staging (staging/develop).

13.1 Kiểm tra chất lượng trước merge

```
# Kiểm tra coding convention cuối cùng
/check-convention src/

# Kiểm tra tiêu chuẩn naming DB
/check-naming src/entity/

# Xác nhận 0 lỗi console
"Kiểm tra xem có lỗi trong console không"
```

13.2 Merge nhánh Staging

Tự động hóa tạo PR → Review → Merge (đối tượng nhánh staging/develop)
/pr-merge

Vai trò nhánh Staging:

- Môi trường tích hợp cho kiểm thử người dùng (UAT)
- Sau khi tất cả nhánh tính năng merge vào staging, tiến hành kiểm thử người dùng thực tế
- Bước xác minh cuối cùng trước khi merge vào nhánh main

14. Kiểm thử người dùng

Người dùng thực tế (DE, stakeholder) trực tiếp xác minh hệ thống trên môi trường staging. Đây là lĩnh vực **phán đoán chuyên môn domain và đánh giá tính khả dụng** mà AI không thể thay thế.

14.1 Sprint Review (1 giờ)

[Sprint Review]

- 30 phút: Demo thời gian thực (chrome-devtools MCP)
 - Không cần chuẩn bị demo riêng – Trình diễn ngay trên môi trường staging
 - Chuyển đổi viewport đa dạng thời gian thực (mobile/tablet/desktop)
 - Kiểm tra network request thời gian thực (chứng minh hoạt động API)
 - Chia sẻ kết quả performance trace
- 30 phút: Feedback DE + Phản ánh ngay lập tức
 - DE: "Phần này hãy thay đổi như thế này"
 - PE: Sửa prompt → AI tái triển khai (5~10 phút)
 - Demo kết quả thay đổi ngay lập tức

14.2 Kiểm thử chấp nhận người dùng (UAT)

DE và stakeholder trực tiếp kiểm thử trên môi trường staging.

Checklist UAT:

- ☐ Xác nhận hoạt động kịch bản nghiệp vụ cốt lõi
- ☐ Xác nhận tính nhất quán dữ liệu (môi trường tương tự dữ liệu thực)
- ☐ Đánh giá tính khả dụng UI/UX
- ☐ Xác nhận edge case và tình huống ngoại lệ
- ☐ Xác nhận cảm nhận hiệu suất (tốc độ phản hồi, tải trang)

14.3 Phản ánh feedback

Issue phát hiện trong kiểm thử người dùng được sửa ngay lập tức hoặc đăng ký vào backlog Sprint tiếp theo.

Loại issue	Đối phó	Thời gian
Có thể sửa ngay	PE sửa prompt → AI tái triển khai	30 phút~2 giờ
Cần thay đổi thiết kế	Sửa blueprint → Phản ánh Sprint tiếp theo	Đăng ký backlog
Thay đổi yêu cầu	Phân tích tác động → DE quyết định ưu tiên	1~2 ngày

14.4 Sprint Retrospective (30 phút)

[Hồi cứu tăng cường AI]

- └ 10 phút: Phân tích tự động dựa trên dữ liệu Sprint (agent sprint-analyzer, Sonnet)
 - └ Mẫu issue lặp lại từ code-review
 - └ Lịch sử chặn security-guidance
 - └ Tần suất vi phạm astra-methodology
 - └ Phân tích mẫu/nhịp độ commit
- └ 10 phút: Thảo luận nhóm (lĩnh vực AI không nắm bắt được)
 - └ Tập trung vào hiểu sai logic domain, vấn đề giao tiếp, v.v.
- └ 10 phút: Tự động hóa cải tiến
 - └ /hookify [Chuyển đổi lỗi lặp đi lặp lại từ hồi cứu thành quy tắc]
 - └ Cập nhật CLAUDE.md
 - └ Cải thiện template prompt Sprint tiếp theo

Ví dụ sử dụng hookify trong hồi cứu:

```
# Quy tắc hóa lỗi lặp đi lặp lại trong Sprint này
/hookify Không để lộ stack trace trong error response
/hookify Không bao gồm thông tin nhạy cảm (mật khẩu, token) trong API response

# Tự động phát hiện dựa trên phân tích cuộc hội thoại (chạy không có tham số)
/hookify
# → Agent conversation-analyzer phát hiện lỗi lặp đi lặp lại từ các cuộc hội thoại gần đây
```

15. Merge vào nhánh Main

Merge nhánh staging đã pass kiểm thử người dùng vào nhánh main (main/master). Chạy Quality Gate cuối cùng (Gate 3) và chuẩn bị release.

15.1 Gate 3: BRIDGE-TIME (Quality Gate cuối cùng)

```
# Kiểm tra chất lượng toàn bộ code
/code-review
/check-convention src/
/check-naming src/entity/

# Xác nhận 0 lỗi console
"Kiểm tra xem có lỗi trong console không"

# DSA kiểm duyệt thiết kế cuối cùng (tính nhất quán toàn bộ màn hình)
# Agent quality-gate-runner chạy tích hợp Gate 1~3 (Sonnet, chỉ đọc)
```

15.2 Tóm tắt tiêu chí pass Quality Gate

Gate	Tiêu chí pass	Biện pháp khi bị chặn
Gate 1	0 cảnh báo security-guidance, 0 từ cấm	Sửa ngay rồi viết lại
Gate 2	0 issue có độ tin cậy cao từ code-review, coverage 70%+	Quyết định fix now / fix later
Gate 2.5	DSA phê duyệt kiểm duyệt thiết kế	Sửa prompt → Tạo lại → Kiểm duyệt lại
Gate 3	0 vi phạm convention/naming, 0 lỗi console	Sửa hàng loạt rồi deploy

15.3 Merge nhánh Main

```
# Merge nhánh Staging → Main
/pr-merge
```

15.4 Tạo sản phẩm release

```
# Tự động tạo hướng dẫn vận hành
/feature-dev "Viết hướng dẫn vận hành dự án tại docs/delivery/operation-manual.md.
Bao gồm quy trình deploy, biến môi trường, điểm giám sát, hướng dẫn xử lý sự cố.
Chưa sửa code."

# Dọn dẹp nhánh
/clean_gone
```

Phụ lục

Phụ lục A: Tham chiếu nhanh công cụ Claude Code

Tình huống	Lệnh/Công cụ sử dụng	Ghi chú
Cấu hình môi trường phát triển toàn cục	/astra-setup	Tự động cấu hình cài đặt toàn cục, MCP, plugin
Hướng dẫn tham chiếu nhanh	/astra-guide	Tóm tắt workflow, lệnh, Quality Gate
Thiết lập ban đầu dự án	/project-init [tên dự án]	Tạo cấu trúc thư mục Sprint 0 + template
Checklist Sprint 0	/project-checklist	Xác minh hoàn thành Sprint 0
Khởi tạo Sprint	/sprint-plan [N]	Tạo prompt map, theo dõi tiến độ, template hồi cứu
Bắt đầu thiết kế tính năng	/feature-dev [mô tả]	Workflow tự động 7 bước
Kiểm tra thuật ngữ chuẩn	/lookup-term [thuật ngữ tiếng Hàn]	Viết tắt tiếng Anh/domain/type
Tra cứu mã quốc tế	/lookup-code [mã]	ISO 3166-1/2, E.164 (quốc gia/vùng/số điện thoại)
Tạo DB entity	/generate-entity [định nghĩa tiếng Hàn]	Dựa trên tài liệu thiết kế DB, Java/TypeScript/SQL
Tạo kịch bản kiểm thử E2E	/test-scenario	Kịch bản E2E dựa trên blueprint, DB, route
Thực thi kiểm thử tích hợp	/test-run	Chạy server + Xác minh tự động Chrome MCP
Kiểm tra coding standard	/check-convention [đối tượng]	Java/TS/RN/Python/CSS/SCSS
Kiểm tra naming DB	/check-naming [đối tượng]	Dựa trên từ điển thuật ngữ chuẩn
Commit	/commit	Tự động tạo message
Tạo PR	/commit-push-pr	Commit+Push+PR hàng loạt
Tự động hóa PR→Review→Merge	/pr-merge	Toàn bộ chu kỳ Commit→PR→Review→Sửa→Merge

Code review	/code-review	5 agent song song
Tạo quy tắc hook	/hookify [mô tả]	Quy tắc ngăn chặn hành vi
Kiểm tra quy tắc hook	/hookify:list	Danh sách quy tắc hiện tại
Tra cứu tài liệu mới nhất	"use context7 - [câu hỏi]"	Tài liệu thư viện
Kiểm tra trình duyệt	chrome-devtools MCP	Snapshot/Screenshot/Hiệu suất
Truy vấn DB	postgres MCP	Thực thi truy vấn trực tiếp

Phụ lục A-2: Tham chiếu nhanh Agent

Agent	Model	Gate	Vai trò
astra-verifier	Haiku	-	Kiểm tra tuân thủ phương pháp luận ASTRA
naming-validator	Haiku	Gate 1/3	Xác minh tiêu chuẩn naming DB (Gate 1: cảnh báo tự động hook, Gate 3: xác minh agent)
convention-validator	Haiku	Gate 1/2	Xác minh coding convention (Gate 1: skill tự động áp dụng, Gate 2: xác minh agent)
blueprint-reviewer	Sonnet	Gate 2	Xác minh chất lượng/nhất quán tài liệu thiết kế
test-coverage-analyzer	Haiku	Gate 2	Phân tích chiến lược/coverage kiểm thử
design-token-validator	Haiku	Gate 2.5	Xác minh tự động tuân thủ hệ thống design token
sprint-analyzer	Sonnet	-	Phân tích tự động tiến độ/hỏi cứu Sprint
quality-gate-runner	Sonnet	Gate 3	Thực thi tích hợp Gate 1~3

Tất cả agent đều **chỉ đọc** (không thể Write/Edit) — chỉ thực hiện phân tích và báo cáo.

Phụ lục B: Hướng dẫn viết Prompt

5 yếu tố của prompt tốt:

- 1. **What (Cái gì):** Mô tả rõ ràng tính năng cần tạo
- 2. **Why (Tại sao):** Mục đích kinh doanh và giá trị cho người dùng
- 3. **Constraint (Ràng buộc):** Ràng buộc kỹ thuật và yêu cầu hiệu suất
- 4. **Reference (Tham chiếu):** Tài liệu thiết kế liên quan, đường dẫn code hiện có
- 5. **Acceptance (Tiêu chí):** Điều kiện hoàn thành và phương pháp xác minh

BAD: "Tạo chức năng thanh toán"
GOOD: /feature-dev "Triển khai module xử lý thanh toán. - Hỗ trợ thanh toán thẻ và chuyển khoản - Tích hợp API PG (KG Inicis) - Tự động retry tối đa 3 lần khi thanh toán thất bại - Tuân theo thiết kế trong docs/blueprints/003-payment/blueprint.md - DB schema tham chiếu docs/database/database-design.md - Viết cả unit test và integration test"

Phụ lục C: Quản lý rủi ro

Rủi ro	Xác suất	Tác động	Chiến lược đối phó
AI hallucination (tạo code sai)	Trung bình	Trung bình	Phát hiện bằng Gate 2 code-review, xác minh API mới nhất bằng context7
Hiểu sai logic nghiệp vụ phức tạp	Trung bình	Cao	Câu hỏi làm rõ bắt buộc ở feature-dev Phase 3, DE tham gia
Sự cố Claude API	Thấp	Cao	Kết hợp môi trường phát triển local, sao lưu thủ công logic cốt lõi
Thuật ngữ chưa đăng ký trong từ điển chuẩn	Trung bình	Thấp	Tạo viết tắt bằng cách kết hợp từ trong standard_words.json
Không phát hiện lỗ hổng bảo mật	Thấp	Cao	9 mẫu security-guidance + Kiểm tra bảo mật cuối cùng song song
Burnout Sprint 1 tuần	Trung bình	Trung bình	AI hấp thụ công việc lặp lại, con người tập trung phán đoán/ra quyết định
Xem nhẹ Scrum ceremony	Trung bình	Trung bình	Giảm thời gian nhưng nhất định duy trì ceremony

Phụ lục D: Cơ sở ước lượng thời gian làm việc của AI Agent

Các ước lượng thời gian làm việc trong tài liệu này dựa trên dữ liệu nghiên cứu thực tế và case study ngành năm 2025~2026.

Dữ liệu nghiên cứu cốt lõi

Nguồn	Phát hiện cốt lõi	Ứng dụng
METR - Time Horizons	Tiêu chuẩn tỷ lệ thành công AI 50%: Claude 3.7 Sonnet ~1 giờ, GPT-5.2 ~6.5 giờ (cuối 2025)	Giới hạn thời gian thực thi tự động cho tác vụ phức tạp
METR - Developer Study	Lập trình viên thành thạo + công cụ AI (Cursor/Claude): chậm hơn 19% khi sử dụng ad-hoc cho tác vụ quy mô 2 giờ	Tầm quan trọng của workflow có cấu trúc
METR - Time Horizon Growth	Thời gian tác vụ tự động AI tăng gấp đôi mỗi 7 tháng, 2024-2025 gấp đôi mỗi ~4 tháng	Ước tính tác vụ tự động 2~4 giờ vào năm 2026

Case study ngành

Nguồn	Phát hiện cốt lõi	Ứng dụng
Faros AI - Best AI Coding Agents 2026	Cursor: Mạnh ở tác vụ nhỏ~trung, vấn đề looping ở refactoring quy mô lớn	Cần chia nhỏ đơn vị tác vụ
Anthropic - Agentic Coding Trends 2026	1 kỹ sư + Claude Code = khối lượng công việc 1 tháng của team truyền thống	Cải thiện đáng kể khi dựa trên spec có cấu trúc
TELUS/Zapier case	TELUS cải thiện 30% tốc độ deploy, tiết kiệm 500K+ giờ / Zapier 89% tổ chức áp dụng	Hiệu quả áp dụng AI cấp tổ chức

Thời gian thực tế theo loại tác vụ (đầu năm 2026, tiêu chuẩn Claude Opus 4.6 / Sonnet 4.5)

Loại tác vụ	AI thực thi tự động	Con người xem xét/sửa	Tổng thời gian
-------------	---------------------	-----------------------	----------------

Phân tích codebase	10~30 phút	30 phút~1 giờ	1~2 giờ
Tạo tài liệu thiết kế	15~30 phút	1~2 giờ	1.5~3 giờ
Triển khai tính năng đơn giản (CRUD)	30 phút~1 giờ	1~2 giờ	2~3 giờ
Triển khai tính năng trung bình (xác thực, tích hợp API)	1~3 giờ	2~4 giờ	4~8 giờ
Triển khai tính năng phức tạp (multi-service, logic nghiệp vụ phức tạp)	3~6 giờ	4~8 giờ	1~2 ngày
Code review tự động	10~15 phút	10~20 phút	20~40 phút
Tạo unit test	Đồng thời với code	30 phút~1 giờ	Đồng thời với code
Tạo UI component	15~30 phút	1~2 giờ (DSA kiểm duyệt)	1.5~3 giờ
Triển khai dựa trên tài liệu thiết kế (có spec)	1~2 giờ	2~4 giờ	3~6 giờ
Triển khai tính năng không có spec (không spec)	4~8 giờ	6~10 giờ	1~2 ngày+

Insight cốt lõi: Chất lượng tài liệu thiết kế (spec) quyết định thời gian làm việc AI một cách quyết định. Tác vụ có thể hoàn thành trong 60 phút với spec tốt, có thể mất hơn 16 giờ mà không có spec. Đây là lý do ASTRA viết blueprint trước.

Giới hạn ước lượng thời gian

- Hiệu suất AI agent có **biên độ lớn tùy theo phiên bản model, kích thước codebase, độ phức tạp domain**
- "Chậm 19%" của nghiên cứu METR dựa trên tiêu chuẩn **sử dụng AI ad-hoc**, trong **workflow có cấu trúc** như ASTRA có thể đạt rút ngắn 30~60% thời gian
- Thời gian tác vụ tự động AI **tăng gấp đôi mỗi ~7 tháng**, nên ước lượng thời gian trong tài liệu này cần **xem xét lại mỗi 6~12 tháng**
- Logic nghiệp vụ phức tạp, quyết định kiến trúc, xác minh chuyên biệt domain vẫn là **nút thắt cổ chai cần phán đoán của con người**

Phụ lục E: Thiết lập dự án Sprint 0

Sprint 0 thiết lập nền tảng dự án trong 1 tuần. Chỉ thực hiện **1 lần** trước tất cả Sprint tính năng.

Step 0.0: Cấu hình môi trường phát triển (toàn cục)

Phạm vi: Đơn vị máy lập trình viên (cấu hình 1 lần, áp dụng cho tất cả dự án)

```
# Bước 1: Thêm marketplace plugin
claude plugin marketplace add https://github.com/ASTRA-TECHNOLOGY-COMPANY-LIMITED/astra-methodology.git

# Bước 2: Cài đặt plugin astra-methodology
claude plugin install astra-methodology@astra

# Bước 3: Thiết lập tự động môi trường phát triển toàn cục (cấu hình toàn cục, MCP server, tự động cài đặt 9 plugin)
/astra-setup
```

Các mục được tự động cài đặt:

- 9 plugin bắt buộc (claude-code-setup, code-review, code-simplifier, commit-commands, feature-dev, frontend-design, hookify, security-guidance, context7)
- 3 MCP server (chrome-devtools, postgres, context7)
- Cấu hình toàn cục (Agent Teams, bypassPermissions, Always Thinking)

Step 0.1: Vision & Backlog (Ngày 1-2)

Xây dựng tầm nhìn dự án thông qua cuộc họp kickoff với DE và viết Product Backlog ban đầu.

```
# Kiểm tra tài liệu mới nhất của công nghệ stack
"use context7 – So sánh WebClient và RestTemplate của Spring Boot 3. Phương pháp được khuyên dùng mới nhất?"

# Phân tích trước tính năng cốt lõi
/feature-dev "Phân tích kiến trúc tổng thể của hệ thống thanh toán trực tuyến
và viết tại docs/blueprints/overview.md. Chưa sửa code thực tế."
```

Step 0.2: Xây dựng Design System (Ngày 2-3) - DSA chủ trì

Chi tiết tham khảo [5. Xây dựng Design System](#).

Xây dựng design token, hướng dẫn style component, hệ thống layout grid.

Step 0.3: Architecture & Standards (Ngày 3-4)

Thực hiện tạo tài liệu thiết kế tính năng cốt lõi (tham khảo [6. Viết Blueprint](#)), viết tài liệu thiết kế DB tập trung (tham khảo [7. Thiết kế cơ sở dữ liệu](#)), viết tài liệu chiến lược kiểm thử (docs/tests/test-strategy.md).

Step 0.4: Cấu hình Guard Rails (Ngày 4-5)

Viết CLAUDE.md + cấu hình quy tắc hookify để thiết lập trước quy tắc chất lượng áp dụng cho toàn bộ Sprint.

```
# Tạo quy tắc tùy chỉnh theo dự án
/hookify Tất cả API endpoint phải bao gồm middleware xác thực
/hookify Sử dụng thư viện logger thay vì console.log
/hookify Sử dụng CSS Variable thay vì giá trị màu hardcoded trong CSS
```

Checklist hoàn thành Sprint 0:

- ☐ Hoàn thành viết Product Backlog ban đầu
- ☐ Hoàn thành xây dựng Design System (design token, hướng dẫn component)
- ☐ Hoàn thành tạo tài liệu thiết kế (MD) theo tính năng cốt lõi và DE phê duyệt
- ☐ Hoàn thành viết tài liệu thiết kế DB tập trung (docs/database/database-design.md)
- ☐ Hoàn thành viết tài liệu chiến lược kiểm thử (docs/tests/test-strategy.md)
- ☐ Hoàn thành viết CLAUDE.md (bao gồm nguyên tắc thiết kế)
- ☐ Hoàn thành cấu hình quy tắc hookify

Xác minh Sprint 0: `/project-checklist`

Phụ lục F: Template dự án

F.1 Cấu trúc thư mục

```
project-root/
├── CLAUDE.md                # Quy tắc AI dự án (cốt lõi!)
├── .claude/
│   ├── hookify.*.local.md  # Quy tắc hookify theo dự án
│   └── settings.json       # Cấu hình Claude theo dự án
├── docs/
│   ├── design-system/      # DSA xây dựng trong Sprint 0
│   │   ├── design-tokens.css
│   │   └── tailwind.config.js
```

```

├── components.md
├── layout-grid.md
├── references/

├── blueprints/                # Tài liệu thiết kế (Living Document)
├──   ├── overview.md
├──   ├── 001-auth/
├──   │   └── blueprint.md
├──   ├── 002-payment/
├──   │   └── blueprint.md

├── database/                  # Tài liệu liên quan CSDL
├──   ├── database-design.md    # Tài liệu thiết kế DB tập trung (toàn bộ bảng/ERD/FK)
├──   ├── naming-rules.md       # Quy tắc naming DB và ánh xạ thuật ngữ chuẩn
├──   └── migration/            # Lịch sử migration
├──       └── v1.0.0.sql

├── tests/                     # Tài liệu liên quan kiểm thử
├──   ├── test-strategy.md      # Chiến lược kiểm thử (định nghĩa phạm vi unit/integration/E2E)
├──   ├── test-cases/           # Đặc tả test case theo tính năng
├──   │   └── sprint-1/
├──   │       └── auth-test-cases.md
├──   ├── test-reports/         # Báo cáo kết quả kiểm thử theo Sprint
├──   │   └── sprint-1-report.md

├── sprints/                   # Tài liệu Sprint
├──   ├── sprint-1/
├──   │   ├── prompt-map.md
├──   │   ├── progress.md
├──   │   └── retrospective.md
├──   └── sprint-2/
├──       └── prompt-map.md

├── delivery/                  # Sản phẩm Release Sprint
├──   ├── operation-manual.md
├──   └── quality-report.md

└── src/                        # Source code

```

F.2 Template hồi cứu Sprint

Sprint [N] Retrospective

Dữ liệu phân tích AI

- Issue lặp lại từ code-review: [Thu thập tự động]
- Số lần chặn security-guidance: [Thu thập tự động]
- Tần suất vi phạm astra-methodology: [Thu thập tự động]

Thảo luận nhóm (lĩnh vực AI không nắm bắt được)

Điều đã làm tốt (Keep)

—

Điều cần cải thiện (Problem)

—

Điều muốn thử (Try)

Biện pháp cải tiến tự động hóa

- /hookify [Quy tắc hóa lỗi lặp đi lặp lại phát hiện trong Sprint này]
- Nội dung cập nhật CLAUDE.md: [Mô tả quy tắc đã thêm]

Phụ lục G: Hiệu quả kỳ vọng

Hiệu quả định lượng

Chỉ số	Mục tiêu ASTRA	Tỷ lệ cải thiện
Chu kỳ Sprint	1 tuần (gia tăng nhỏ, phản hồi nhanh)	Rút ngắn 50% chu kỳ lặp
Thời gian ceremony mỗi Sprint	4 giờ	Giảm 67%
Nhân lực đầu vào	4~5 người	Giảm 50%
Tỷ lệ tuân thủ coding standard	95%+ (bắt buộc tự động)	Cải thiện +30%
Thời gian code review	20~40 phút (tự động)	Rút ngắn 85~90%
Đối phó thay đổi yêu cầu	1~2 ngày	Rút ngắn đáng kể so với 2 tuần+ truyền thống
Thời gian lập trình	Rút ngắn 40~60% so với truyền thống	Dựa trên nghiên cứu METR
Thời điểm phát hiện lỗi hỏng bảo mật	Thời điểm viết code	Chuyển đổi hậu kiểm → tiền kiểm
Tỷ lệ cập nhật tài liệu thiết kế	100% (Living Document)	Cải thiện +70%
Xác minh Definition of Done	Tự động (Gate 1-3)	Chuyển đổi thủ công → tự động

Hiệu quả định tính

1. **Tập trung vào bản chất Scrum:** Giảm thời gian ceremony → Tập trung vào "chuyển giao giá trị"
2. **Cải thiện văn hóa review:** Loại bỏ tranh luận style/tiêu chuẩn → Chuyển thành diễn đàn thảo luận logic nghiệp vụ
3. **Hiệu quả thực tế của hồi cứu:** "Sẽ cải thiện" → "Bắt buộc bằng quy tắc hookify"
4. **Tăng mức tham gia DE:** Trở thành đối tác thực sự của dự án nhờ demo thời gian thực và phản ánh ngay lập tức
5. **Giảm nợ kỹ thuật:** Tích hợp chất lượng tại thời điểm viết, loại bỏ từ gốc "để sau sửa"
6. **Dễ dàng chuyển giao kiến thức:** Tối thiểu hóa chi phí bàn giao nhờ Living Document

Phụ lục H: Hiệu quả chi phí

Chi tiết tham khảo [2.6 Hiệu quả chi phí](#).